

Malware Concepts: Rootkit

The first three guides in this series (Trojan, Worm, Virus) were all fundamentally about **how malware spreads**. A rootkit is about something entirely different: **how malware hides, once it's already there**. This is a genuinely important shift in focus to notice — a rootkit isn't really competing with Trojan/Worm/Virus as a delivery category at all; it's frequently the *thing a Trojan delivers*, operating at a deeper level of the system than nearly anything else covered in this series, specifically to make detection and removal as difficult as possible.

1. What a Rootkit Actually Is — The Foundation

A rootkit is a collection of software tools specifically designed to gain and **maintain privileged (root/administrator-level) access to a system while actively concealing its own presence** — from the user, from standard system utilities, and often from security software itself.

The single defining trait: STEALTH AT A PRIVILEGED SYSTEM LEVEL.

A rootkit's core purpose is not necessarily to DO damage directly (though it often enables further malicious activity) - its purpose is to ensure that whatever access or malware it's protecting remains INVISIBLE to normal detection and inspection methods for as long as possible.

Why the name "Rootkit"?

"Root" in the name directly references Unix/Linux root-level (administrator-equivalent) access - though rootkits exist across Windows, macOS, Linux, and even firmware/hardware level targets, the name reflects the PRIVILEGE LEVEL the technique targets, not a Unix-specific limitation.

2. Categories of Rootkit by Operating Level

The deeper into the system a rootkit operates, the harder it generally is to detect and remove — this hierarchy is worth understanding as a spectrum, not just a list:

Rootkit Category	Operating Level / Behavior
User-mode rootkit	Operates at the same privilege level as normal applications; intercepts and modifies API calls to hide files, processes, and registry entries from standard tools - the LEAST stealthy category, generally detectable by security tools operating at a deeper system level.
Kernel-mode rootkit	Operates within the operating system's kernel itself, the same privilege level as the OS core - able to intercept and manipulate system calls directly, making it invisible to security tools that only operate at the user-mode level.
Bootkit	Infects the boot process itself (the Master Boot Record or UEFI firmware), loading BEFORE the operating system does - directly extends the boot sector virus concept from the Virus guide into a much more sophisticated, persistence-focused technique.
Firmware/hardware rootkit	Operates at the firmware level of a device itself (network card, hard drive controller, BIOS/UEFI) - the DEEPEST and most difficult category to detect or remove, since it can survive a complete OS reinstall entirely.

3. Core Concealment Techniques

Concealment Technique	Description
Process hiding	Intercepting process-listing calls so the rootkit's own running process never appears in Task Manager, <code>ps</code> output, or equivalent tools.
File/directory hiding	Similarly intercepting file-listing calls to hide the rootkit's own files from directory browsing and file search utilities.
Registry/config hiding	Hiding persistence-related registry keys or configuration entries from standard inspection tools.
Network connection hiding	Concealing active network connections the rootkit or its associated malware maintains, directly relevant when a rootkit is protecting an active RAT connection (see the RAT guide).
Log tampering	Modifying or deleting system logs that would otherwise record the rootkit's installation or activity.

The Unifying Mechanism

The unifying mechanism across all of these: a rootkit generally doesn't hide by "deleting evidence" in a simple sense — it works by intercepting the very

system calls that legitimate tools rely on to *ask* the operating system what's running or what files exist, and simply returning a filtered, dishonest answer. This is precisely why operating at the kernel level (Section 2) is so much more effective than user-mode operation — a kernel-mode rootkit can lie to *every* tool asking the question, including tools that would otherwise be trusted to detect it.

4. How Rootkits Are Typically Installed

Rootkits are typically installed through one of the following paths:

- **Bundled as the payload of a Trojan** (directly connecting back to that guide's Section 3 payload categories) - the user is deceived into running an installer that includes rootkit components alongside its apparent function.
 - **Delivered via successful exploitation of a system vulnerability**, where the exploit's post-exploitation stage installs the rootkit to maintain access.
 - **Installed by an attacker who has already achieved administrative/root access through some other means** (credential theft, privilege escalation), specifically as a PERSISTENCE mechanism - directly relevant to your Linux Privilege Escalation series, since a rootkit is often exactly what an attacker installs immediately after successfully escalating to root, to guarantee they don't need to repeat that escalation chain on their next visit.
-

5. Real-World Impact

Impact	Description
Extended dwell time	Because detection is specifically what a rootkit is designed to defeat, systems compromised with a well-implemented rootkit can remain compromised for months or years without detection, dramatically increasing the total damage and data exposure over the compromise's lifetime.
Protects and enables other malware	A rootkit rarely operates alone; it frequently exists specifically to shield an active RAT connection, a persistent backdoor, or ongoing data exfiltration from detection, meaning its real-world impact is often best measured through whatever it's protecting rather than standalone.

Extremely difficult and costly remediation	Kernel-mode and firmware-level rootkits in particular can require a full OS reinstall, or in the firmware case, specialized reflashing or physical hardware replacement, to fully remediate.
---	--

6. Detection Considerations

- **Standard, in-system detection tools are fundamentally UNRELIABLE against a well-implemented rootkit**, since the rootkit can directly manipulate the very calls those tools rely on - this is the central, defining detection challenge for this entire malware category.
- **Offline/external detection** (booting from trusted, verified media and inspecting the system from OUTSIDE its normal running state) bypasses the rootkit's ability to lie to in-system tools, and is the most reliable detection approach available.
- **Behavioral/network-level anomaly detection** (unusual outbound traffic, unexpected resource usage patterns) can sometimes surface a rootkit's presence indirectly, even when direct in-system inspection fails.
- **Cross-view detection** (comparing what different tools/methods report about the same system state, and investigating any discrepancy) is a specific, well-established rootkit-hunting technique built directly on the fact that a rootkit typically cannot successfully lie to every possible inspection method simultaneously.
- **Firmware/hardware-level integrity verification** (measured boot, TPM-based attestation) is specifically necessary to address the deepest category from Section 2, since OS-level tools have no visibility into that layer at all.

7. Prevention

- Maintain rigorous **patch management**, since rootkit installation frequently depends on first exploiting a vulnerability to gain the privileged access required to install the rootkit itself.
- Enable **Secure Boot and UEFI-level integrity verification** specifically to defend against the bootkit and firmware-level categories from Section 2.
- Apply the principle of **least privilege** broadly, since nearly every rootkit category in this guide requires administrative/root-level access to install in

the first place - directly connecting back to your Linux Privilege Escalation series' entire premise.

- Deploy security tooling capable of **kernel-level and offline/external inspection** specifically, rather than relying solely on standard in-system antivirus that a sufficiently sophisticated rootkit can potentially blind or deceive.
- Treat rootkit discovery as grounds for a **FULL system reinstall from a known-clean source** in most cases, rather than attempting in-place removal, given how thoroughly a rootkit can compromise the trustworthiness of the running system's own reporting about itself.

Quick Reference Cheat Sheet

Defining Trait

Maintains privileged access while actively **CONCEALING its own presence** from standard detection methods.

Categories by Operating Level — Least → Most Stealthy/Severe

Category	Summary
User-mode rootkit	Intercepts API calls at application level.
Kernel-mode rootkit	Operates within the OS kernel itself.
Bootkit	Infects MBR/UEFI and loads before the OS.
Firmware/hardware rootkit	Operates below the OS entirely; can survive a full OS reinstall.

Core Concealment Techniques

Process hiding, file/directory hiding, registry/config hiding, network connection hiding, log tampering — all achieved by intercepting and lying to the system calls detection tools rely on.

Typical Installation Path

```
Trojan payload
↓
Exploitation post-exploitation stage
↓
```

Attacker-installed persistence mechanism

↓

Privileged access maintained and concealed

Key Detection Principle

In-system tools are unreliable against a rootkit by design.

Prefer **OFFLINE/EXTERNAL inspection** and cross-view comparison.
